Using the Segger J-Link Debugger with the Portenta Breakout

This tutorial teaches you how to set up the board with the Segger J-link debugger.

Author · Benjamin Dannegård Last revision · 09/25/2024

Overview

This tutorial will show you how to debug an Arduino sketch using the Portenta H7, Portenta Breakout board and the Segger J-link device. Using the Arduino IDE we will build a version of the sketch so that it can be used in Segger's Ozone debugger.

Goals

- How to debug an Arduino sketch
- How to connect the Portenta breakout to the Segger J-link device
- How to use the Segger J-link and Portenta with the Ozone debugger

Required Hardware and Software

- Portenta H7 board

USB-C® cable (USB-C® to USB-A cable)

Arduino IDE 1.8.10+ or Arduino IDE 2.0+

J-link adapter

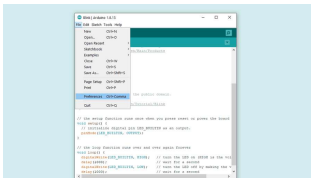
Segger J-link device

Segger Ozone

Instructions

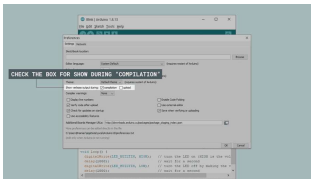
1. Setting up with the Arduino IDE

First, if you haven't done it yet, install the Arduino IDE and connect your Portenta H7. When uploading a sketch to your Arduino board with the Arduino IDE, it will build an .ELF file of the sketch. We will need this file to debug in Ozone in the next steps. To easily find the file path of the .ELF file, we can enable the show verbose output option in the Arduino IDE. To do this, open up the preferences under **File > Preferences** in the Arduino IDE.



Preferences in Arduino IDE

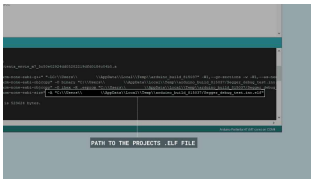
When you have the preferences window open, look for the **Show verbose output during: compilation** option and make sure that the checkbox is ticked.



Arduino IDE preference window

Now we are ready to upload the script that we want to debug. If you don't have a sketch to test, you can use the example sketch found in the **Conclusion** section of this tutorial.

When we upload the sketch with the Arduino IDE, we need to know where the .ELF file will be saved. Build your project in the Arduino IDE and highlight the output directory; it should look for example like `C:\Users\profile\AppData\Local\Temp\arduino_build_815037`. Note down the path for easier access in the next step.



The .elf file path in Arduino IDE

2. Connecting the Boards and Device

First, connect the Portenta H7 to the Breakout. To begin using the J-link device with the Portenta H7 and Breakout, you will need a J-link adapter, sold separately. The cable for the adapter needs to be connected as indicated in the illustration below, with the cable

side, connect the cable going away form the J-link device on the adapter. Now you just need to connect the USB cable from the J-link to your computer and the USB cable from the Portenta H7 that is on the Breakout to your computer.

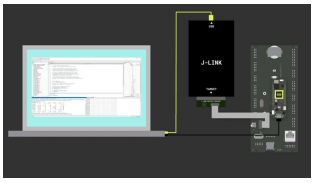
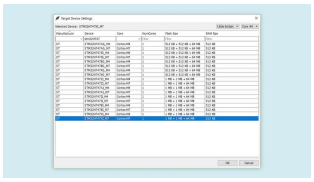


Illustration of connection

3. Using the Setup with Segger Ozone

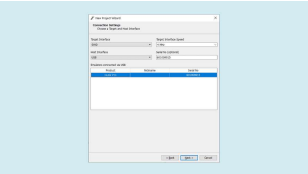
Download and install Segger's Ozone debugger. If you are on Windows, make sure to also download the J-Link Software and Documentation Pack for Windows.

When starting Ozone, make sure to enter the correct CPU into the settings box. The Portenta H7 uses the **STM32H747XI**; you can then choose between the M4 and M7 core on the Portenta.



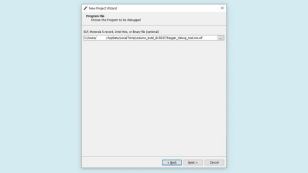
Segger Ozone J-Link cpu settings

Continue to the next step. Here you need to change the **Target Interface** to **SWD**. The Portenta H7 does not support JTAG. Then select your J-link device in the list of emulators and



Segger Ozone J-Link connection settings

Now you get to the window that asks you to select the program to be debugged, this is where you load the project's .ELF file with the temporary output path that we noted before. Navigate to the correct directory, and select the .ELF file.



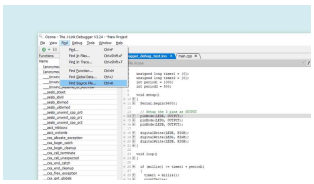
Temporary .ELF file location

In the 'optional settings' dialog, set both options 'Initial PC' and 'Initial Stack Pointer' to 'Do not set' as it would skip the Arduino bootloader, otherwise this may prevent the sketch from running correctly.

Segger Ozone J-Link connection optional settings

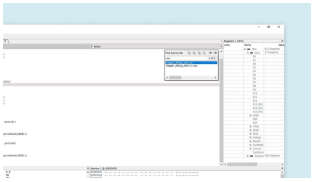
When the setup is finished, Ozone will open the file containing the main function. You will note that this is not the .ino sketch you wrote since this is an abstraction layer generated

we need to go to **Find > Find source file** in the top toolbar.



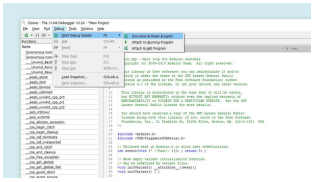
Find source file in Ozone

In the little window that appears, type ".ino". You should now be able to see the file, select the file and open it in Ozone.



Find source file window

Now you are ready to start debugging. Simply go to **Debug > Download & Reset Program** to start debugging your sketch: you can add breakpoints , inspect variables, halt the execution and more.



Start debugging in Ozone

For more information about the features present in the Ozone debugger, please go here.

Conclusion

In this tutorial, you learned how to connect your Portenta H7 and

also went through how to create a file with Arduino IDE that can be debugged in Ozone. And eventually how to use the Ozone debugger to debug an Arduino sketch.

Example Sketch

```
int counter1 = {0};
int counter2 = {0};

unsigned long timer1 =
unsigned long timer2 =
int period1 = 1000;
int period2 = 500;

void setup()
{

  // Set the LED pins u
  pinMode(LEDR, OUTPUT)
  pinMode(LEDG, OUTPUT)
  pinMode(LEDB, OUTPUT)

  digitalWrite(LEDR, HI
  digitalWrite(LEDG, HI
  digitalWrite(LEDB, HI
}

void loop()
{

  if (millis() >= timer
  {
    timer1 = millis();
    counter1++;
```

anything wrong, you can edit this page here.

Was this article helpful?